

Correction de la série de TD ASD N°5 (Les chaînes de caractères)

EXERCICE 1

Algorithme chaîneCarre

Variables

ch : chaîne de caractère
i, L, n : entier
carré : booléen

Début

```

Ecrire("donner une chaîne")
Lire(ch)
L ← long(ch)
n ← L div 2
si (L mod 2 ≠ 0) alors
    Ecrire("la chaîne n'est pas carré")
sinon
    carré ← vrai
    i ← 1
    répéter
        si (ch[i] ≠ ch[i+n]) alors
            carré ← faux
        finSi
        i ← i+1
    jusqu'à (carré = faux ou i = n+1)
    si (carré) alors
        Ecrire("la chaîne est carré")
    sinon
        Ecrire("la chaîne n'est pas carré")
    Finsi
finSi

```

Fin

EXERCICE 2

Algorithme nbMotsChaîne

Variables

ch, chM, chML : chaîne de caractère
i, L, nbMot : entier
trouve : booléen

Début

```

Répéter
    Ecrire("donner une chaîne non vide")
    Lire(ch)
Jusqu'à (long(ch) ≠ 0)
L ← long(ch)
chML ← "" //le mot le plus long
nbMot ← 0
i ← 1
répéter
    trouve ← faux
    chM ← "" //le mot courant
    tantque (i ≤ L et Majus(ch[i]) ≥ 'A' et Majus(ch[i]) ≤ 'Z') faire
        trouve ← vrai
        chM ← chM + ch[i]
        i ← i+1
    finTQ

```

Ou bien

tantque (i ≤ L et ch[i] ≠ ' ' et ch[i] ≠ '.' et ch[i] ≠ ',') faire

```

    si (trouve) alors //ou : si (chM ≠ "")
        nbMot ← nbMot + 1
        si (long(chM)>long(chML)) alors
            chML ← chM
        finSi
    finSi
    i ← i+1
Jusqu'à (i>=L+1)
Ecrire(" le mot le plus long est : ",chML)
Ecrire(" le nombre de mots est : ",nbMot)

Fin

En langage C

#include<stdio.h>
#include<string.h>
#define max 101
void main(){
    char ch[max];
    char chML[max]="";
    int L,nbMots=0; //ou bien
    gets(ch); fgets(ch, max, stdin);
    L=strlen(ch);

    for(int i=0;i<L;i++)
    {
        char chM[max]="";
        int trouve=0;
        while(i<L && toupper(ch[i])>='A' && toupper(ch[i])<='Z')
        {
            trouve=1;
            strncat(chM,ch+i,1); //ou bien : chM[strlen(chM)]=ch[i];
            i++;
        }
        if (trouve)
        {
            chM[strlen(chM)]='\0';
            puts(chM);
            nbMots++;
            if(strlen(chM)>strlen(chML))
                strcpy(chML,chM);
        }
    }
    printf("\nle mot le plus long est %s: ",chML);
    printf("\nle nbre de mots est %d:",nbMots);
}

```

EXERCICE 3

Algorithme lettreChaine

Variables

ch : chaine de caractère
i, Ln, nbocc, prepos, dernierpos : entier
L : caractère

Début

Répéter

Ecrire("donner une chaine non vide")
Lire(ch)

Jusqu'à (long(ch)≠0)

Répéter

Ecrire("donner une lettre")
Lire(L)

Jusqu'à (Majus(L)>='A' et Majus(L)<= 'Z')

```

nbocc ← 0
premos ← 0
dernierpos ← 0
Ln ← long(ch)
pour i de 1 à Ln faire
    si (ch[i]=L) alors
        nbocc ← nbocc + 1
        dernierpos ← i
        si (nbocc=1) alors
            premos ← i
        finSi
    finSi
Finpour

```

```

Ecrire(" le nombre d'apparition de la lettre: ",L, " dans la chaine: ",ch, " est: ",nbocc)
si (premos≠0) alors //Ou bien // si (nbocc≠0) alors
    Ecrire(" la première position de la lettre: ",L, " dans la chaine: ",ch, " est: ",premos)
    Ecrire(" la dernière position de la lettre: ",L, " dans la chaine: ",ch, " est: ", dernierpos)
sinon
    Ecrire(" la lettre: ",L, " est absente dans la chaine: ",ch)
finSi

```

//supprimer toutes les occurrences de la lettre L dans la chaine ch

```

i ← premos
tantque (i<=Ln) faire
    si (ch[i]=L) alors
        j ← i
        tantque (j<=Ln-1) faire
            ch[j] ← ch[j+1]
            j ← j+1
        finTQ
        Ln ← Ln-1
    sinon
        i ← i+1
    finsi
finTQ
Ecrire(" la nouvelle chaine est : ")
pour i de 1 à Ln faire
    Ecrire(ch[i])
Finpour
Fin

```

2^{ème} sol

```

i ← premos
tantque (i<= long(ch)) faire
    si (ch[i]=L) alors
        Effacer(ch, i, 1)
    Sinon
        i ← i+1
    finsi
finTQ
Ecrire(" la nouvelle chaine est : ",ch)

```

3^{ème} sol

```

tantque (pos(L,ch)≠-1) faire
    Effacer(ch, pos(L,ch), 1)
finTQ
Ecrire(" la nouvelle chaine est : ",ch)

```

4^{ème} sol

```

Pour i de 1 à nbocc faire
    Effacer(ch, pos(L,ch), 1)
finPour

```

EXERCICE 4

Algorithme suppChaine

Variables

ch1, ch2 : chaine de caractères
i, j, L1, L2 : entier
trouve : booléen

Début

```

Lire(ch1,ch2)
L1 ← long(ch1)
L2 ← long(ch2)
i ← 1
tantque (i<=L1) faire
    j ← 1
    trouve ← faux

```

```

tantque (j<=L2 et non(trouve)) faire
    si (ch1[i]=ch2[j]) alors
        trouve←vrai
    finsi
    j←j+1
finTQ
si (trouve) alors
    pour j de i à L1-1 faire
        ch1[j]←ch1[j+1]
    finpour
    L1← L1-1
Sinon
    i←i+1
finsi
finTQ
Ecrire(" la nouvelle chaine est : ")
pour i de 1 à L1 faire
    Ecrire(ch1[i])
Finpour

```

Fin

EXERCICE 5

Algorithme motAnagrammes

Variables Mot1, mot2 : chaine de caractère

i, L, j : entier

test: booléen

Début

Ecrire("donner deux mots ")

Lire(mot1,mot2)

Si (long(mot1)=0 et long(mot2)=0) **alors**

Ecrire(" les deux mots sont anagrammes ")

Si non

Si (long(mot1) ≠ long(mot2)) **alors**

Ecrire(" les deux mots ne sont pas anagrammes ")

Sinon

i←0

L← long(mot1)

répéter

i←i+1

j←0

test← faux

répéter

j←j+1

si (mot1[i]=mot2[j]) **alors**

test← vrai

mot2[j]←'*'

finsi

jusqu'à(test=vrai ou j=L)

jusqu'à(test =faux ou i=L)

si (test) **alors**

Ecrire(" les deux mots sont anagrammes ")

sinon

Ecrire(" les deux mots ne sont pas anagrammes ")

Finsi

Finsi

Finsi

Fin

2^{ème} sol

```

i←1
tantque (i<= long(ch1)) faire
    si (pos(ch1[i] , ch2)≠-1) alors
        Effacer(ch1, i, 1)
    Sinon
        i←i+1
    finsi
finTQ
Ecrire(" la nouvelle chaine est : ",ch1)

```

2^{ème} sol

```

i←1
tantque (i<=L et pos(ch1[i] , ch2)≠-1) faire
    ch2[pos(ch1[i] , ch2)]←'*'
    i←i+1
finTQ
si (i=L+1) alors
    Ecrire(" les deux mots sont anagrammes ")
sinon
    Ecrire(" les 2 mots ne sont pas anagrammes ")
Finsi

```

EXERCICE 6

Constante globale p=10 //taille du tableau

Constante globale n=5 //taille maximale des chaînes de caractères

Procédure saisirT (Var T: TabCh)

Variable i: entier

Début

Pour i de 1 à p **faire**

 Ecrire("donner une chaîne de caractère")

répéter

 Lire(T[i])

Jusqu'à (long(T[i])<=n)

finPour

Fin

Procédure afficherT (T: TabCh)

Variable i: entier

Début

Pour i de 1 à p **faire**

 ecrire(T[i])

finPour

Fin

Procédure triChaine (var ch: chaîne)

Variable j, ln: entier

 échange : booléen

 x : caractère

Debut

ln←long(ch)

répéter

 échange←faux

pour j de 1 à ln-1 **faire**

si (ch[j]>ch[j+1]) **alors**

 x ← ch[j]

 ch[j]← ch[j+1]

 ch[j+1]←x

 échange←vrai

finSi

finpour

jusqu'à (échange=faux)

Fin

Procédure triTab (Var tab: TabCh)

Variable i: entier

 échange : booléen

 ch : chaîne de caractères

DEBUT

répéter

 échange←faux

pour i de 1 à p-1 **faire**

si (tab[i]>tab[i+1]) **alors**

 ch ← tab[i]

 tab[i]← tab[i+1]

 tab[i+1]←ch

 échange←vrai

finSi

finpour

jusqu'à (échange=faux)

Fin

Algorithme Principal

Type : TabCh : Tableau [1..p] de chaîne de caractère

Variables

Tab : TabCh

i : entier

Début

saisirT(Tab)

pour i de 1 à p **faire**

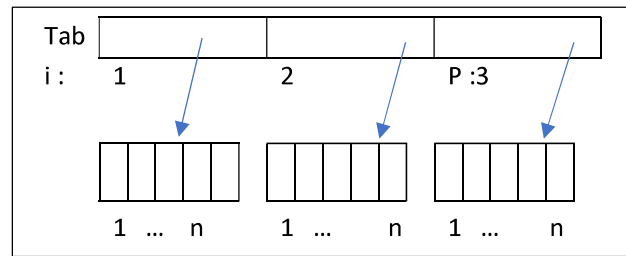
 triChaine (Tab[i])

fin pour

triTab (Tab)

afficherT(Tab)

Fin



En langage C

```
#include<stdio.h>
#include<string.h>
#define p 3
#define n 15

void afficherTab(char Tab[p][n])
{
    printf("\n-----\n");
    for(int i=0;i<p;i++)
        printf("%s\t",Tab[i]);
    printf("\n-----\n");
}

void main(){
    char Tab[p][n];
    char ch[n];
    char x;
    int echange;
    //chargement du tableau
    for(int i=0;i<p;i++)
    {
        printf("donner la chaîne %d:\n",i+1);
        gets(Tab[i]);
    }

    //Affichage du tableau
    printf("\nle tableau est :\n");
    afficherTab(Tab);

    //tri de chaque case
    for(int i=0;i<p;i++)
    {
        strcpy(ch,Tab[i]);
        int ln=strlen(ch);
        do{
            echange=0;
            for(int j=0;j<ln-1;j++)
            {
                if (ch[j]>ch[j+1])
                {
                    x=ch[j];
                    ch[j]=ch[j+1];
                    ch[j+1]=x;
                    echange=1;
                }
            }
        }while(echange==1);
        strcpy(Tab[i],ch);
    }

    //Affichage du tableau
    printf("\nle tableau apres tri de chaque case est :\n");
```

```

afficherTab(Tab);

//tri de tout le tableau
do{
    echange=0;
    for(int i=0;i<p-1;i++)
    {
        if (strcmp(Tab[i],Tab[i+1])>0)
        {
            strcpy(ch,Tab[i]);
            strcpy(Tab[i],Tab[i+1]);
            strcpy(Tab[i+1],ch);
            echange=1;
        }
    }
}while(echange==1);

//Affichage du tableau
printf("\nle tableau trie est :\n");
afficherTab(Tab);

printf("\n");
}

```